Python Programming Assignment

Practical Exercises & Interview Preparation Questions

Data Structures | Loops | Functions | File Handling | Exception Handling | Conditional Statements

Instructions

• Attempt all Practical Questions by writing and running working Python code (.py file per topic, or one notebook per section).

• Interview Questions should be answered in your own words; where a question includes a coding task, submit runnable code as well.

• Do not use built-in shortcuts where a question explicitly asks you to avoid them — the goal is to practice the underlying logic.

• Add comments to your code explaining key steps.

1. Conditional Statements

Practical Questions

1. A retail store in Bhopal gives a discount based on the bill amount: 20% if the bill is above ₹5,000, 10% if between ₹2,000–₹5,000, and no discount below ₹2,000. Write a program that takes the bill amount as input and prints the final payable amount.
2. Write a program that takes a student's marks (out of 100) and prints the grade: A (≥90), B (75–89), C (60–74), D (40–59), Fail (below 40).
3. Write a program to check whether a given year is a leap year.
4. A city's electricity board charges ₹5/unit for the first 100 units, ₹7/unit for the next 100 units, and ₹10/unit beyond 200 units. Write a program to calculate the electricity bill for a given number of units.
5. Write a program that accepts three numbers and prints the largest and the smallest using conditional statements only (no built-in max/min).
6. Write a program to check whether a number is positive, negative, or zero, and whether it is odd or even.

Interview Questions

1. What is the difference between `if-elif-else` and multiple separate `if` statements? Give an example where using separate `if` statements instead of `elif` changes the output.
2. Explain Python's ternary (conditional) expression with an example. When would you prefer it over a full `if-else` block?
3. What does Python treat as "falsy" in a conditional check? List at least five falsy values.
4. Write a one-line program using a ternary expression to check if a number is even or odd.
5. What is the difference between `is` and `==` when used inside a conditional statement? Give an example where they produce different results.
6. How does Python's `match-case` statement (structural pattern matching) differ from a chain of `if-elif` statements? When would you use one over the other?

2. Loops

Practical Questions

7. Write a program to print the multiplication table of a number entered by the user.
8. A warehouse in Indore tracks daily shipment counts for a week in a list. Write a program using a loop to calculate the total, average, maximum, and minimum shipments for the week.
9. Write a program to check whether a given number is prime using a `for` loop.
10. Write a program that prints the Fibonacci series up to `n` terms using a `while` loop.
11. Write a program to reverse a string using a loop (without using slicing or built-in reverse functions).
12. Write a program that uses nested loops to print the following pattern for `n` rows, where row 1 has one number, row 2 has two numbers, and so on (e.g. for n=4: '1' / '1 2' / '1 2 3' / '1 2 3 4').
13. Write a program to count the number of vowels, consonants, digits, and spaces in a given sentence using a loop.

Interview Questions

7. What is the difference between `break`, `continue`, and `pass` in a loop? Give one example where using `continue` instead of `break` changes the final output.
8. Explain the `for...else` construct in Python. When does the `else` block execute, and when is it skipped?
9. What is the difference between iterating over a list with a `for` loop versus using `enumerate()`? When would you prefer `enumerate()`?
10. How does a `while True` loop with a `break` condition differ from a standard `for` loop in terms of use cases? Give a real-world scenario where `while True` is more appropriate.
11. What are list comprehensions, and how do they relate to `for` loops? Rewrite the following loop as a single list comprehension: it builds a list called result by looping x from 0 to 9, and appending x*x whenever x is even.
12. What is the time complexity consideration when using nested loops? Explain with an example of an $O(n^2)$ operation.

---

3. Data Structures (List, Tuple, Set, Dictionary)

Practical Questions

14. A logistics company in Pune stores order IDs in a list. Write a program to remove duplicate order IDs while preserving their original order.
15. Write a program that takes a dictionary of employee names and salaries and prints the name of the employee with the highest salary.
16. Write a program to merge two dictionaries. If a key exists in both, sum their values.
17. Write a program using a set to find the common elements between two lists of customer IDs from two different branches (Bhopal and Indore).
18. Write a program that takes a list of tuples, where each tuple contains a product name and its price, and sorts the list by price in descending order.
19. Write a program to find the second-largest number in a list without using the `sorted()` or `max()` built-in functions.

20. Write a program that counts the frequency of each word in a given sentence and stores the result in a dictionary.

Interview Questions

13. What is the difference between a list and a tuple in Python? Why would you choose a tuple over a list in a given scenario?
14. Why are tuples hashable but lists are not? What practical consequence does this have (e.g., using them as dictionary keys)?
15. Explain the difference between shallow copy and deep copy in the context of nested lists or dictionaries. Give an example where a shallow copy causes an unexpected bug.
16. How does a Python `set` achieve average O(1) lookup time? Why can't you store unhashable types like lists inside a set?
17. What is the difference between `dict.get()` and directly accessing a dictionary with `dict[key]`? When would `get()` prevent a runtime error?
18. What are dictionary and set comprehensions? Write a dictionary comprehension that maps each number from 1 to 5 to its square.
19. How is a Python list implemented internally (array-based vs linked list), and what does that mean for the time complexity of insertion at the beginning versus the end?

---

4. Functions

Practical Questions

21. Write a function `calculate_gst(amount, rate)` that calculates and returns the GST amount and the total payable amount for a purchase.
22. Write a function `is_palindrome(text)` that checks whether a given string is a palindrome, ignoring case and spaces.
23. Write a function that accepts a variable number of numeric arguments using `*args` and returns their sum, average, and maximum.
24. Write a function `generate_invoice(customer, **items)` that accepts a customer name and any number of item=price keyword arguments, and prints a formatted invoice with the total.
25. Write a recursive function to calculate the factorial of a number.
26. Write a function that takes a list of numbers and returns a new list containing only the prime numbers, using a helper function to check primality.
27. Write a lambda function to sort a list of dictionaries (each representing a product with 'name' and 'price') by price.

Interview Questions

20. What is the difference between positional arguments, keyword arguments, `*args`, and `**kwargs`? Describe an example function signature that uses all four together, and the order they must appear in.
21. Explain Python's default argument behavior with mutable default values (e.g., `def f(x, lst=[])`). Why is this considered a common bug, and how do you fix it?
22. What is the difference between a function argument passed by "assignment" in Python versus "pass by reference" or "pass by value" in other languages? Illustrate with a list and an integer example.

23. What are closures in Python? Write a small example of a function that returns another function which remembers a value from the enclosing scope.
24. What is the difference between `*args` unpacking and `**kwargs` unpacking when calling a function versus when defining one?
25. Explain the difference between a regular function and a generator function (using `yield`). Why would you use a generator instead of returning a full list?
26. What is a decorator in Python? Write a simple decorator that prints the execution time of the function it wraps.

---

5. File Handling

Practical Questions

28. Write a program that writes a list of employee records (name, department, salary) to a text file, one record per line, comma-separated.
29. Write a program that reads the text file created above and prints only the records of employees from the 'Sales' department.
30. Write a program to count the number of lines, words, and characters in a text file.
31. Write a program that appends a new order entry to an existing CSV file `orders.csv` using the `csv` module.
32. Write a program that reads a CSV file of sales data (city, product, amount) and prints the total sales amount for each city.
33. Write a program that copies the content of one text file into another, converting all text to uppercase in the process.
34. Write a program that reads a JSON file containing customer details and prints the names of all customers whose city is 'Bhopal'.

Interview Questions

27. What is the difference between opening a file in `'w'`, `'a'`, `'x'`, and `'r+'` modes? Give a scenario for each.
28. Why is using the `with open(...) as f:` context manager preferred over manually calling `open()` and `close()`? What happens if an exception occurs mid-write without using `with`?
29. What is the difference between `read()`, `readline()`, and `readlines()`? When would reading a large file line-by-line with `readline()` (or by iterating the file object) be preferable to `read()`?
30. How do you handle a `FileNotFoundError` gracefully when trying to open a file that may not exist?
31. What is the difference between text mode and binary mode (`'r'` vs `'rb'`) when opening a file? Give an example of a file type that must be opened in binary mode.
32. How would you efficiently process a very large log file (several GB) in Python without loading the entire file into memory at once?

---

6. Exception Handling

Practical Questions

35. Write a program that accepts two numbers from the user and divides them, handling `ZeroDivisionError` and `ValueError` (for non-numeric input) with appropriate messages.
36. Write a program that reads a value from a dictionary of product prices using a product name entered by the user, handling the case where the product does not exist using a `try-except` block.
37. Write a program that validates a user's age input, raising a custom exception `InvalidAgeError` if the age is negative or greater than 120.
38. Write a program that opens a file and processes its content, using `try-except-else-finally` so the file is always closed and a success message is shown only if no error occurred.
39. Write a program that processes a list of transaction amounts (as strings) and converts each to a float, logging any entries that cannot be converted instead of stopping the program.
40. Write a program that attempts an API-style operation (simulate with a function that randomly raises `ConnectionError`) and retries up to 3 times before giving up, using exception handling.

Interview Questions

33. What is the difference between `except Exception as e` and a bare `except:`? Why is using a bare `except:` generally discouraged?
34. Explain the purpose of the `finally` block. Does `finally` execute even if the `try` block contains a `return` statement? Does it execute if the exception is unhandled?
35. What is the difference between `try-except-else` and just putting the else code inside the `try` block? Why does the distinction matter?
36. How do you create and raise a custom exception in Python? What is the benefit of inheriting from `Exception` versus a more specific built-in exception class?
37. What is exception chaining (`raise ... from ...`) and why is it useful for debugging?
38. What is the difference between handling an error with `try-except` versus validating input beforehand with conditional checks (the "EAFP vs LBYL" philosophy)? Which does Python favor, and why?
39. If multiple `except` blocks match a raised exception's type hierarchy, how does Python decide which block to execute? Does order matter?